Name: Nelson Puthuraja Nadar **Roll No.:** 21 **Class/Batch:** S.E / INFT-B / B-1

7. Laboratory Exercise
A. Procedure:
a. Answer the following:

1) What is a Jenkins pipeline?

A Jenkins Pipeline is a suite of plugins and tools that enables the implementation and automation of Continuous Integration and Continuous Delivery, often abbreviated as CI/CD, processes within the Jenkins automation server. It allows developers and DevOps teams to define the entire build, test, and deployment workflow as code, using a structured script known as a *Jenkinsfile*. This approach ensures that the software delivery process is consistent, repeatable, and version controlled.

In a Jenkins Pipeline, the various stages of the software development lifecycle are defined programmatically. These stages typically include tasks such as compiling source code, executing automated tests, performing code quality checks, packaging the application, and deploying the final build to different environments. By defining these tasks in a pipeline, organizations can automate the process from code commit to deployment, reducing manual intervention and minimizing the risk of human error.

Another important aspect of Jenkins Pipelines is that they support complex workflows and allow multiple steps to be executed sequentially or in parallel. Pipelines can also handle conditional execution, error handling, and integration with external tools and services. Since pipelines are stored as code within the project's repository, they promote better collaboration, transparency, and maintainability of the build and deployment process.

Overall, Jenkins Pipelines provide a flexible and powerful way to automate software delivery workflows. By defining the CI/CD pipeline as code, teams can maintain consistency across builds, quickly identify issues, and continuously improve the reliability and efficiency of the development process.

2) What are the different ways to write a Jenkins pipeline?

There are primarily two ways to write and define a Jenkins Pipeline. These approaches are known as **Declarative Pipeline** and **Scripted Pipeline**. Both methods use the Groovy-based domain specific language supported by Jenkins, but they differ in structure, flexibility, and complexity.

The **Declarative Pipeline** is a more structured and simplified way of defining a pipeline. It uses a predefined syntax that organizes the pipeline into clear sections such as *pipeline*, *agent*, *stages*, and *steps*. This approach is designed to make pipeline creation easier and more readable, especially for beginners or teams that prefer a standardized format. Declarative pipelines enforce a well defined structure, which helps maintain consistency and

reduces the chances of configuration errors. Because of its clarity and simplicity, the declarative style is widely recommended for most CI/CD pipeline implementations.

The **Scripted Pipeline**, on the other hand, provides greater flexibility and control. It is written entirely using Groovy scripting and allows developers to implement more complex logic, dynamic workflows, and advanced programming constructs. Scripted pipelines are useful when the pipeline requires conditional logic, loops, custom functions, or highly dynamic behavior that cannot easily be expressed in the declarative format. However, because it relies more heavily on Groovy scripting, it can be more complex to write and maintain.

Both declarative and scripted pipelines can be defined within a **Jenkinsfile**, which is typically stored in the source code repository alongside the application code. Storing the pipeline definition in version control ensures traceability, collaboration among team members, and easier management of pipeline configurations.

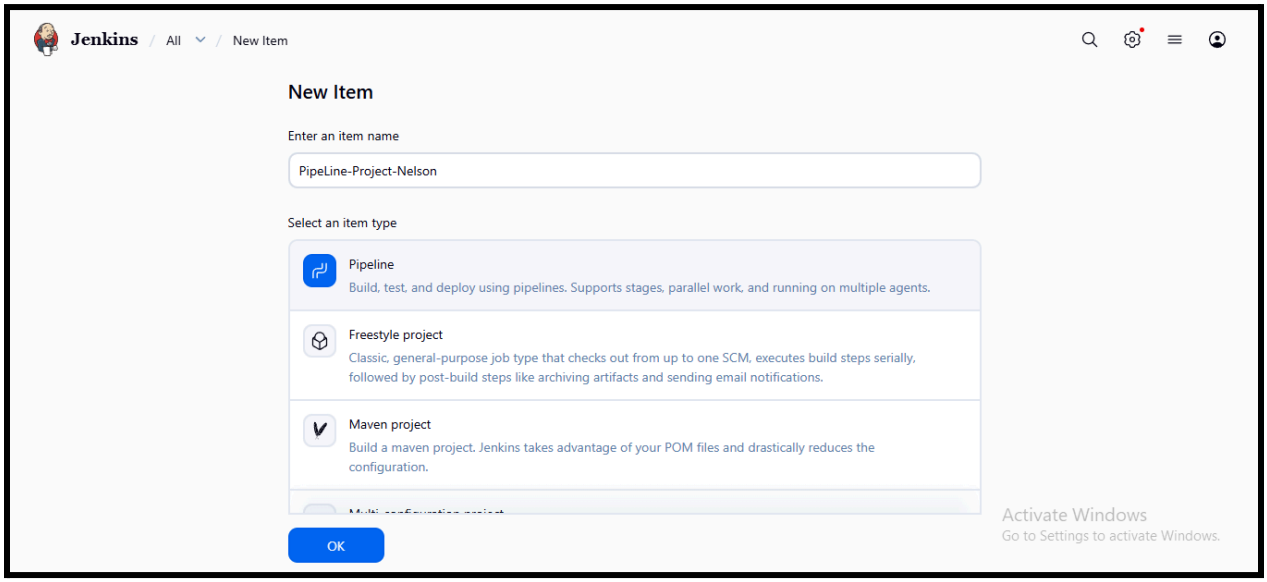
In summary, Jenkins pipelines can be written using either the declarative or scripted approach. The declarative pipeline offers simplicity, structure, and readability, while the scripted pipeline provides flexibility and advanced customization for complex workflows.

B. Execute following and attach screenshots:

1. Create and build pipeline project with pipeline script

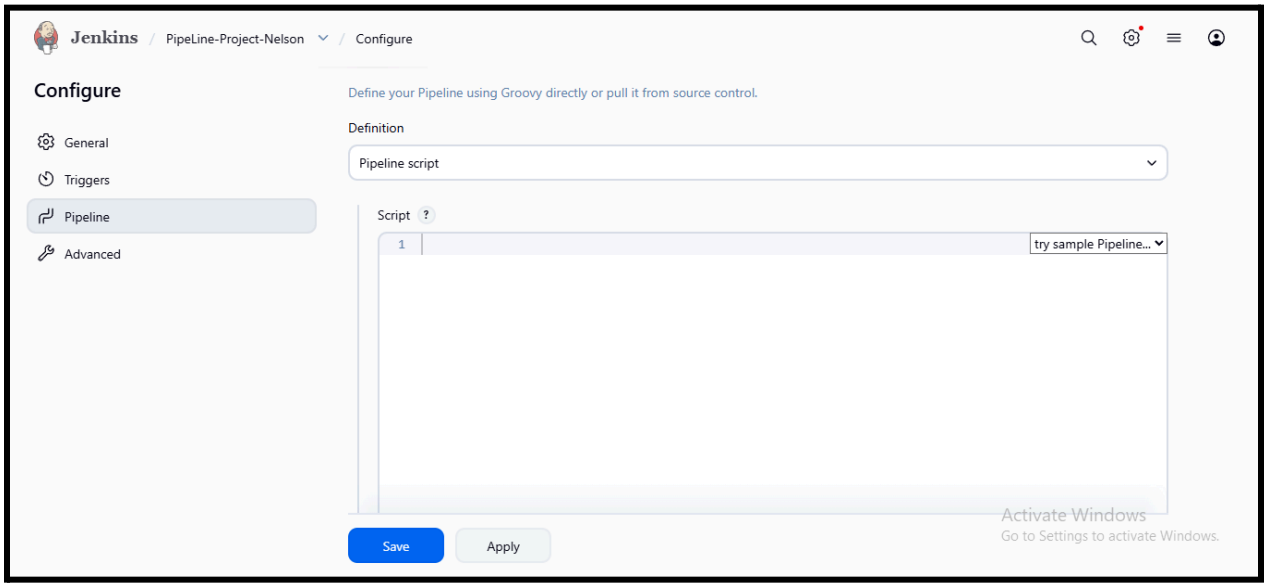
- **Project with Hello World pipeline script**

Create a Pipeline



Name: Nelson Puthuraja Nadar **Roll No.:** 21 **Class/Batch:** S.E / INFT-B / B-1

Configure the Pipeline to Pipeline Script



Using the sample Pipeline Script of HelloWorld

Study Reference

• **pipeline** → Defines the overall Jenkins pipeline structure. It is the main block where the entire CI/CD workflow is written and organized.

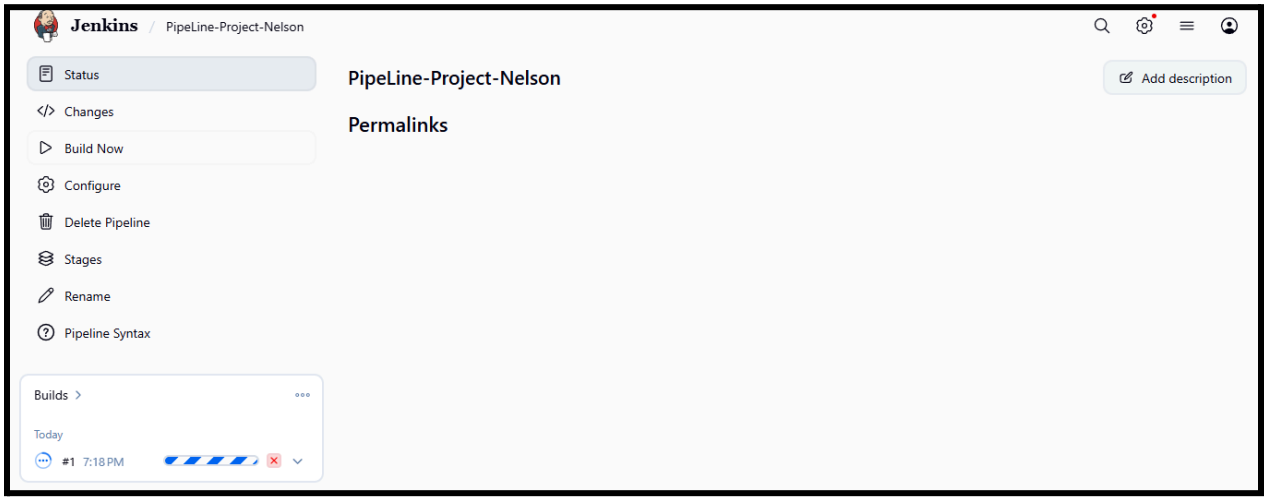
• **agent any** → Specifies that the pipeline can run on **any available Jenkins agent (node)**. Jenkins will automatically choose any machine that is free to execute the pipeline.

• **stage** → Represents a **specific phase of the pipeline**, such as Build, Test, or Deploy. Each stage helps organize the workflow into clear logical steps.

• **steps** → Contains the **actual commands or actions** that Jenkins must execute inside that stage. This is where scripts, shell commands, or Jenkins functions are written.

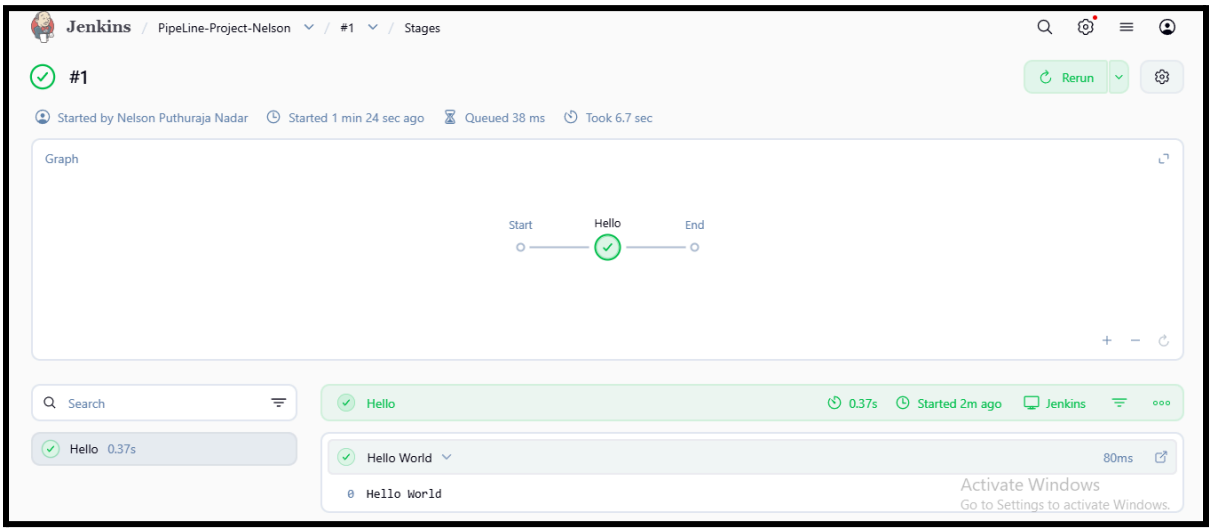
• **echo** → Prints a **message to the Jenkins console output**. It is mainly used for displaying status messages or debugging information during pipeline execution.

Apply -> Save -> Build the Pipeline



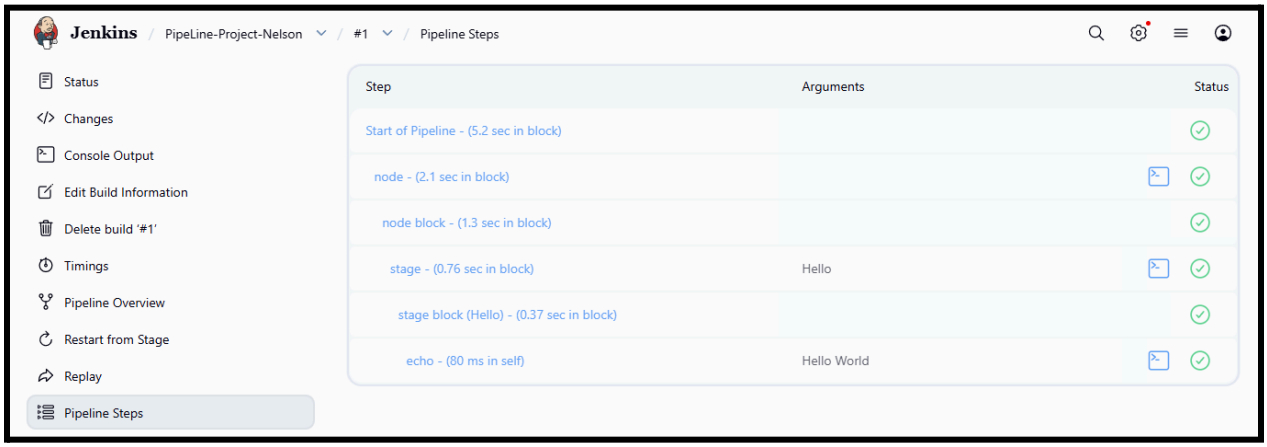
Verifying Console Output

Viewing Pipeline Overview Visualization



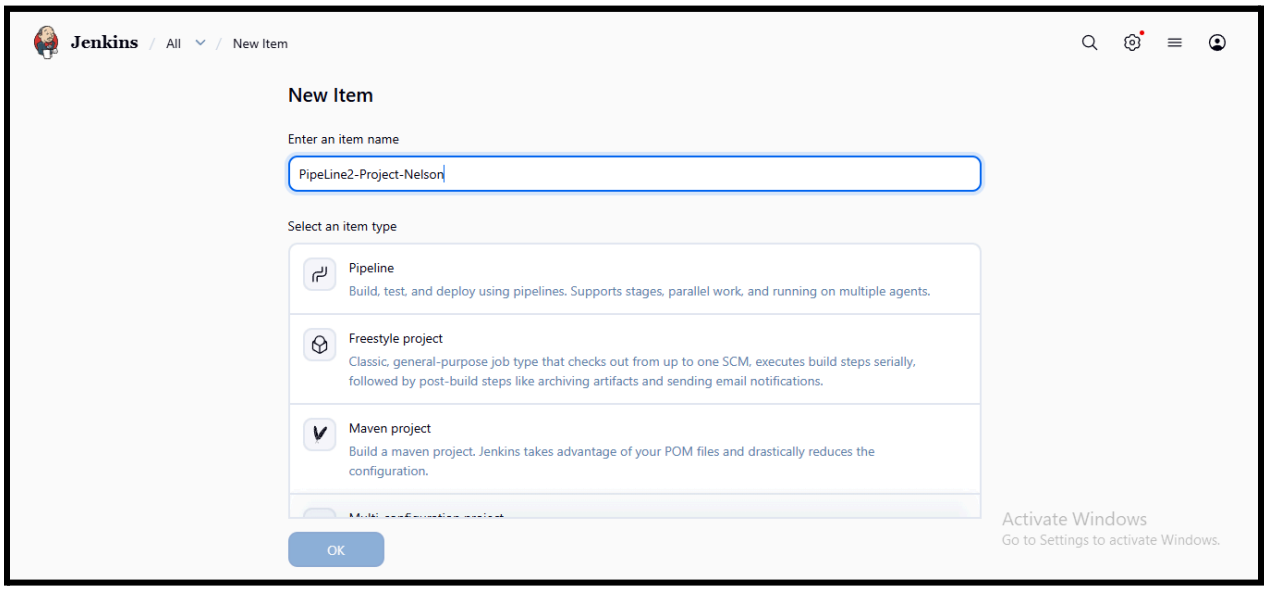
Name: Nelson Puthuraja Nadar **Roll No.:** 21 **Class/Batch:** S.E / INFT-B / B-1

Pipeline Steps



- **Project with your own pipeline script**

Create a Pipeline



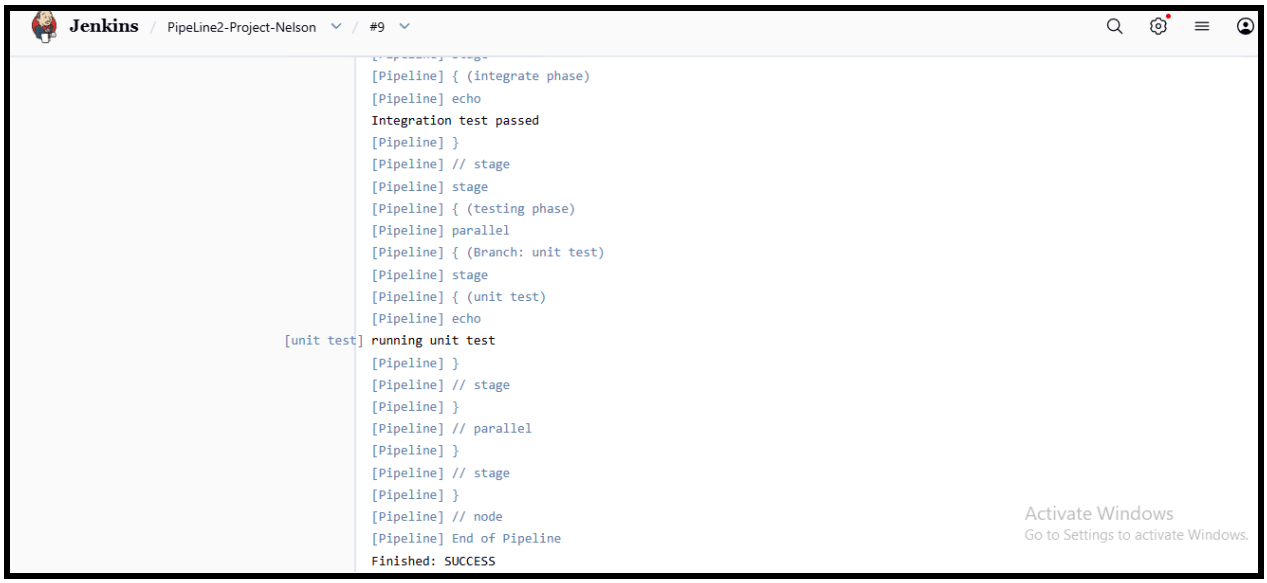
Define Pipeline Script -> Pipeline Code

Name: Nelson Puthuraja Nadar **Roll No.:** 21 **Class/Batch:** S.E / INFT-B / B-1

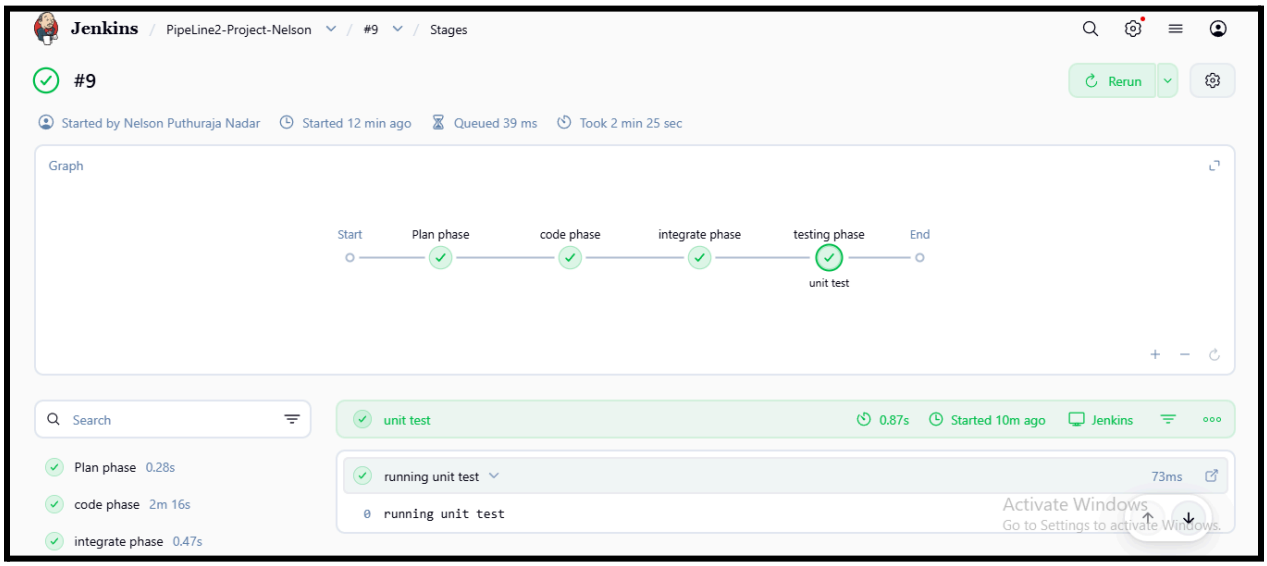
Apply -> Save -> Build the Pipeline

Console Output

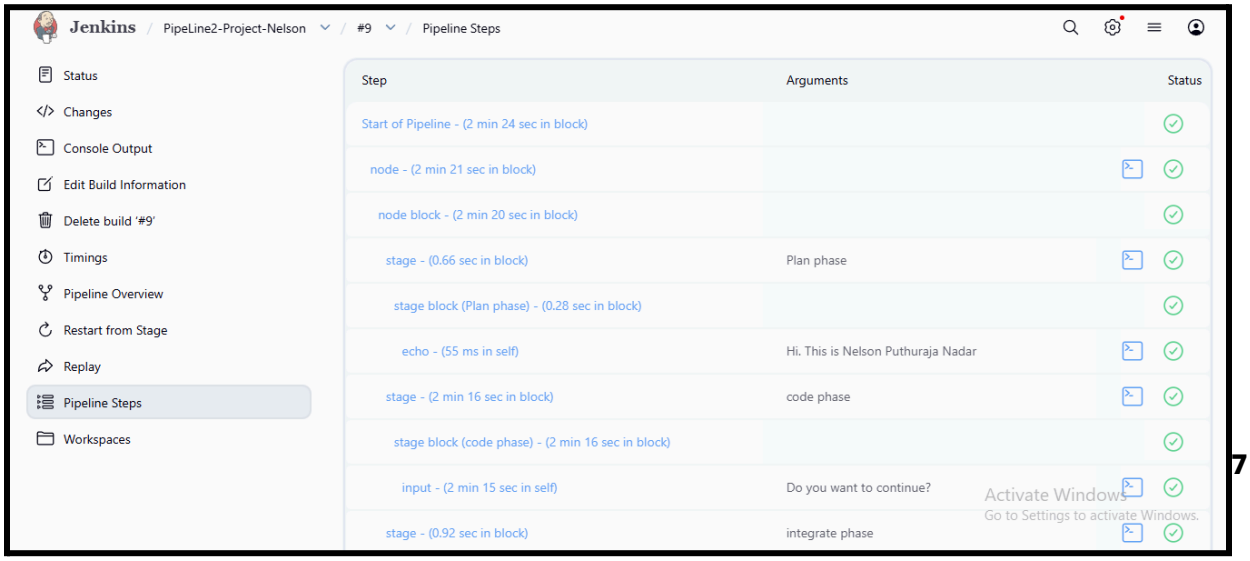
Name: Nelson Puthuraja Nadar **Roll No.:** 21 **Class/Batch:** S.E / INFT-B / B-1



Pipeline Overview



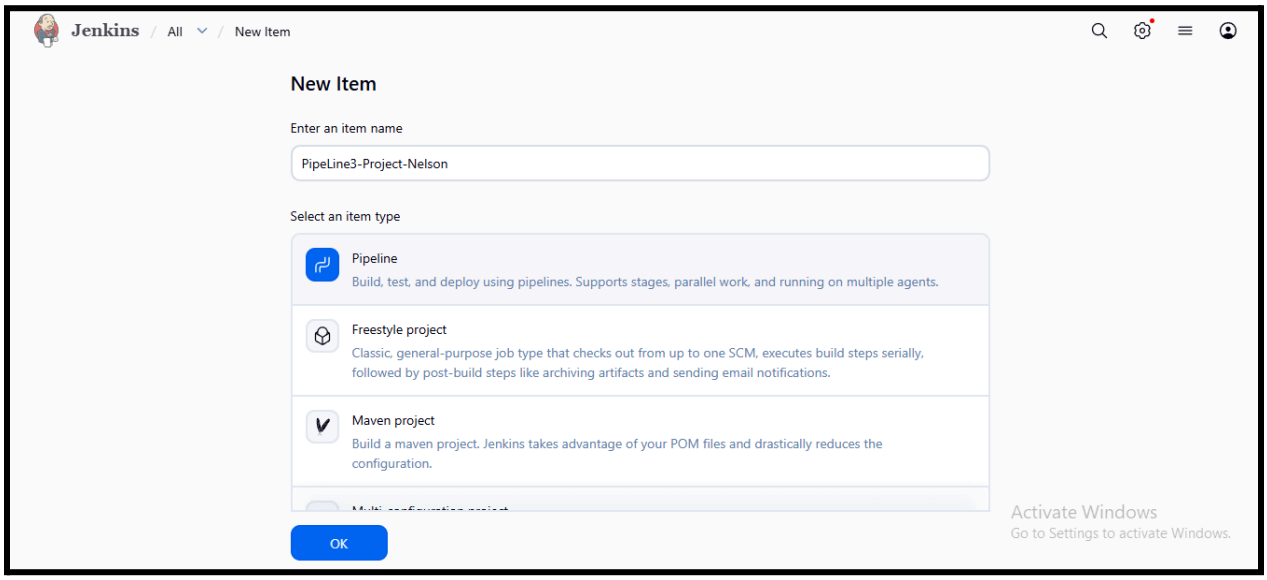
Pipeline Steps



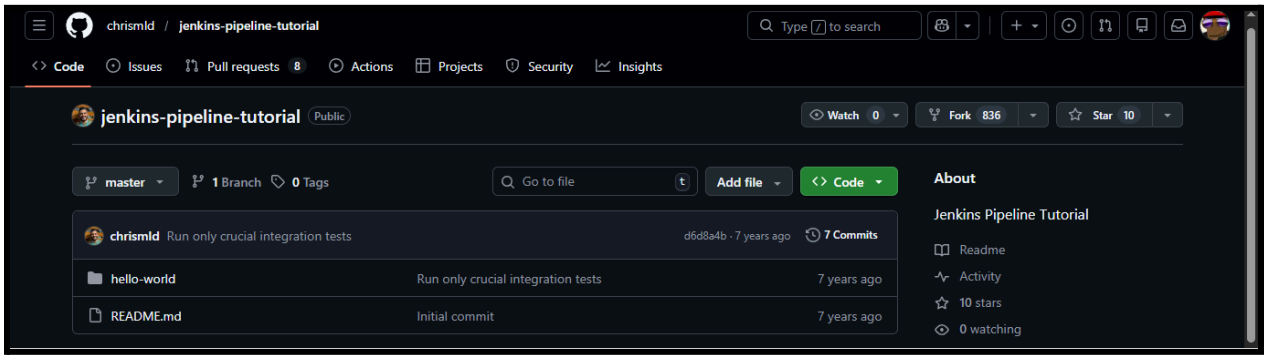
2. Create and build pipeline project with Git

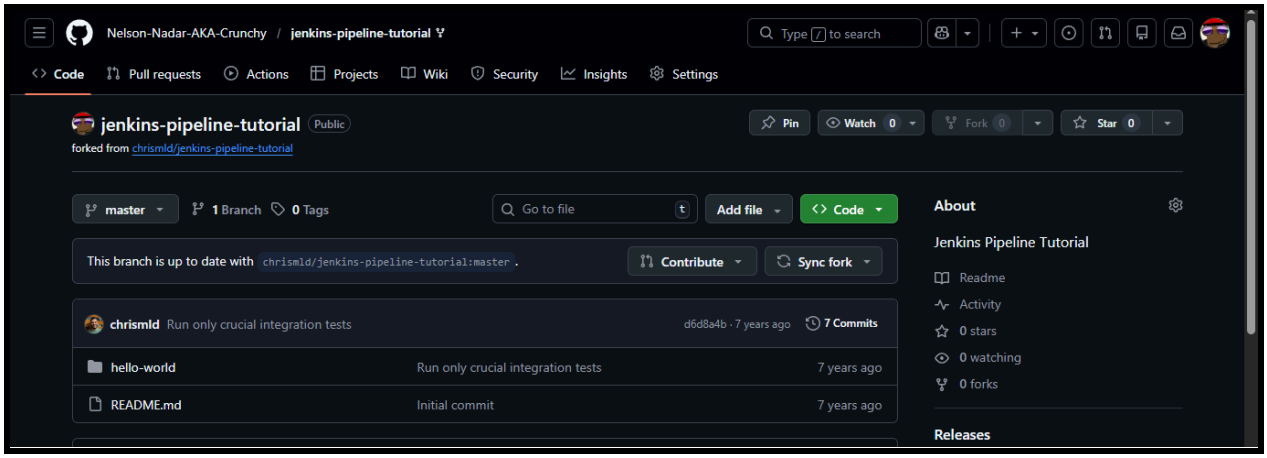
- **Fork repository on GitHub & Create pipeline project with pipeline script from SCM**

Creation of new Pipeline Project

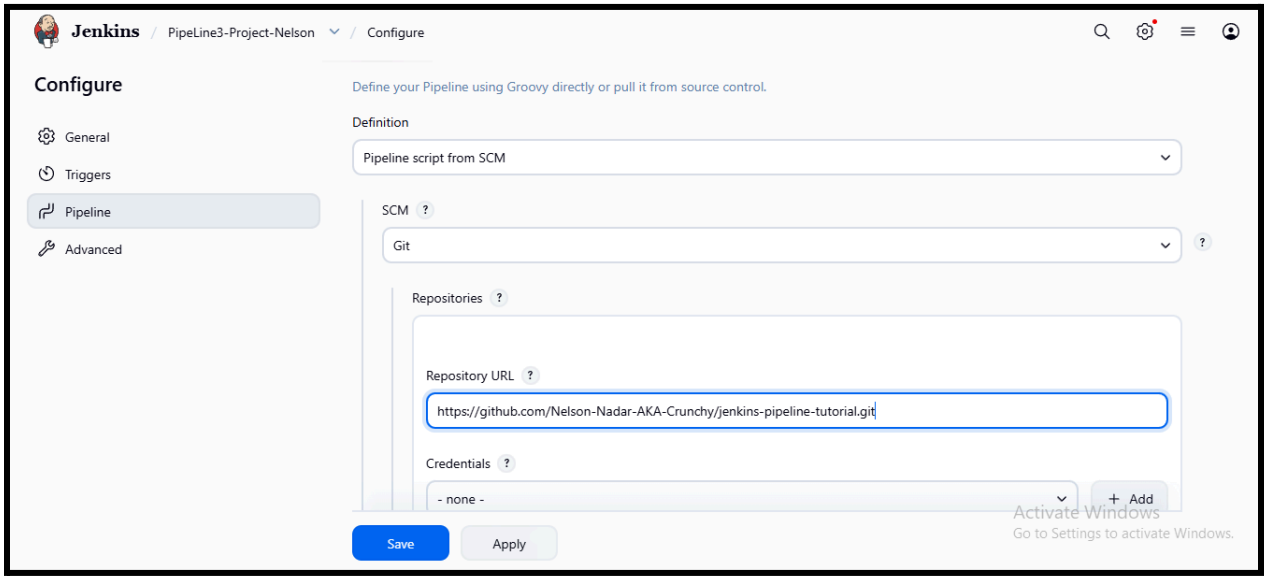


Forked GitHub Repository

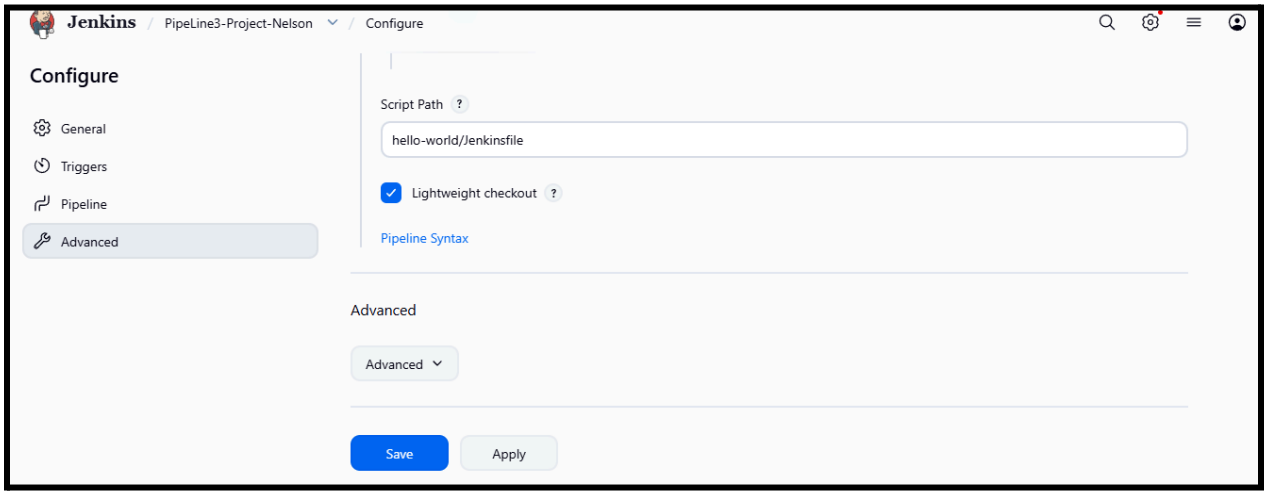




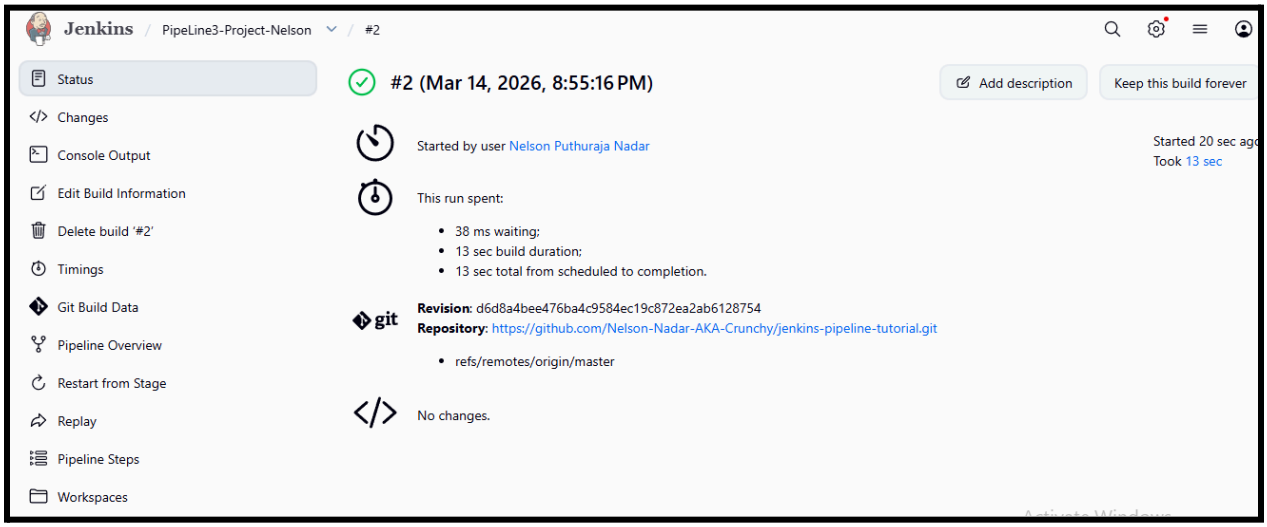
Configure Definition to Pipeline Script from SCM -> SCM to Git -> Paste Repository URL



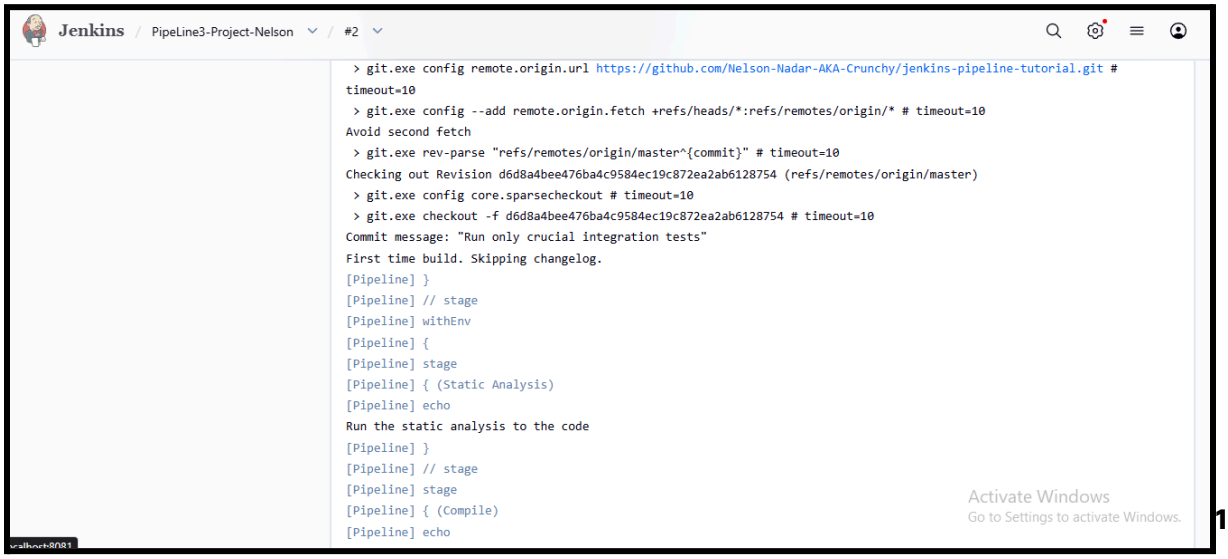
Add Script Path

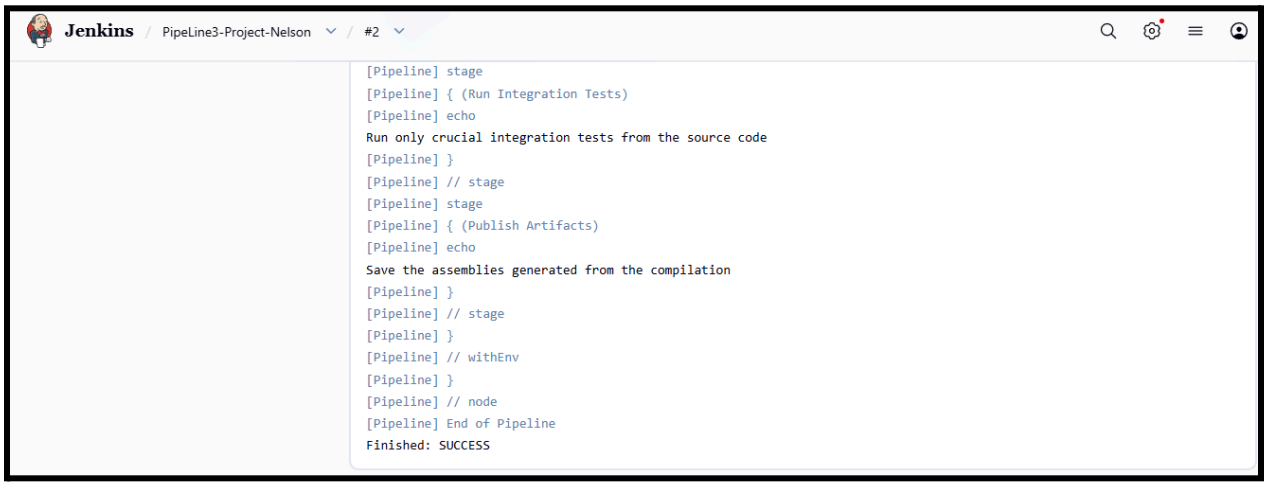


Apply -> Save -> Build the Pipeline

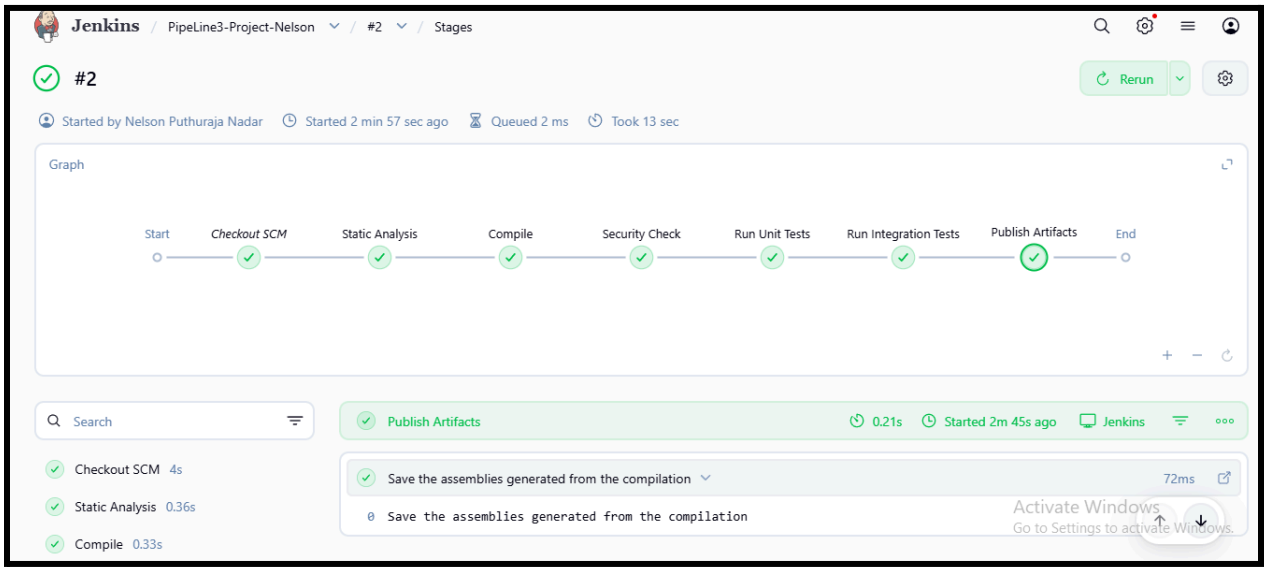


Console Output





Pipeline Visualization



Pipeline Steps

Jenkins / PipeLine3-Project-Nelson / #2 / Pipeline Steps

- Status
- Changes
- Console Output
- Edit Build Information
- Delete build '#2'
- Timings
- Git Build Data
- Pipeline Overview
- Replay
- Pipeline Steps
- Workspaces

Step	Arguments	Status
Start of Pipeline - (12 sec in block)		
node - (10 sec in block)		
node block - (10 sec in block)		
stage - (5.1 sec in block)	Declarative: Checkout SCM	
stage block (Declarative: Checkout SCM) - (4.5 sec in block)		
checkout - (4.3 sec in self)		
withEnv - (4.9 sec in block)	GIT_BRANCH, GIT_COMMIT, GIT_URL	
withEnv block - (4.6 sec in block)		
stage - (0.78 sec in block)	Static Analysis	
stage block (Static Analysis) - (0.36 sec in block)		

Activate Windows
Go to Settings to activate Windows.

Jenkins / PipeLine3-Project-Nelson / #2 / Pipeline Steps

echo - (0.19 sec in self)	Run the static analysis to the code	
stage - (1 sec in block)	Compile	
stage block (Compile) - (0.33 sec in block)		
echo - (73 ms in self)	Compile the source code	
stage - (0.74 sec in block)	Security Check	
stage block (Security Check) - (0.23 sec in block)		
echo - (73 ms in self)	Run the security check against the application	
stage - (0.63 sec in block)	Run Unit Tests	
stage block (Run Unit Tests) - (0.22 sec in block)		
echo - (73 ms in self)	Run unit tests from the source code	
stage - (0.63 sec in block)	Run Integration Tests	

Activate Windows
Go to Settings to activate Windows.

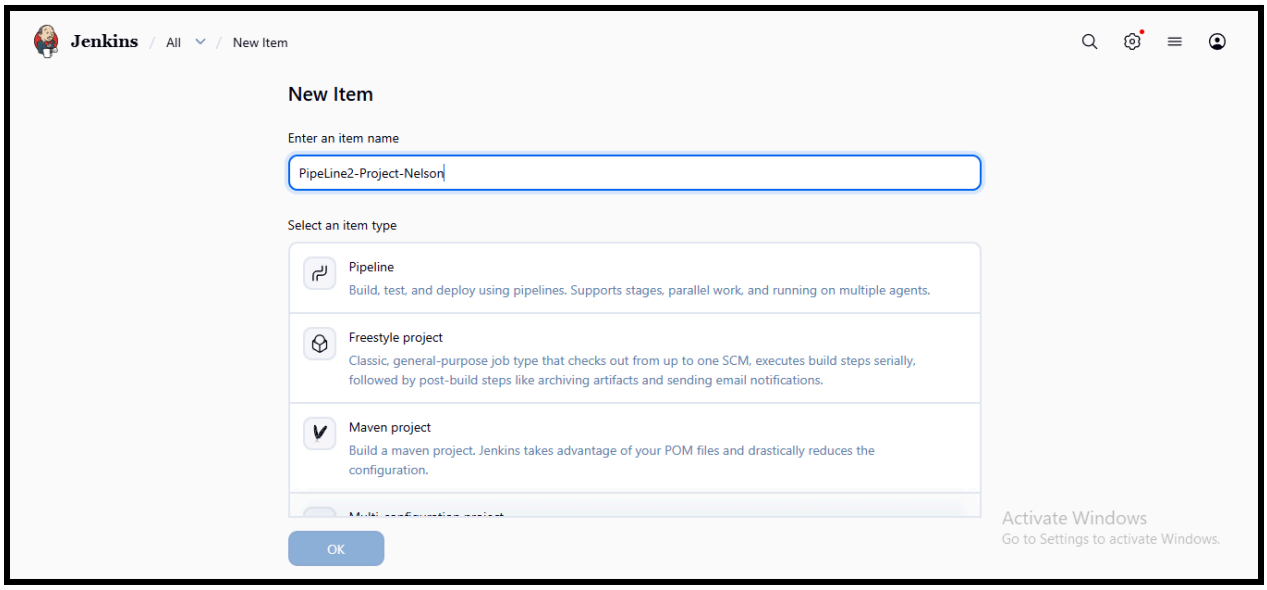
Jenkins / PipeLine3-Project-Nelson / #2 / Pipeline Steps

stage - (0.63 sec in block)	Run Unit Tests	
stage block (Run Unit Tests) - (0.22 sec in block)		
echo - (73 ms in self)	Run unit tests from the source code	
stage - (0.63 sec in block)	Run Integration Tests	
stage block (Run Integration Tests) - (0.23 sec in block)		
echo - (72 ms in self)	Run only crucial integration tests from the source code	
stage - (0.47 sec in block)	Publish Artifacts	
stage block (Publish Artifacts) - (0.21 sec in block)		
echo - (72 ms in self)	Save the assemblies generated from the compilation	

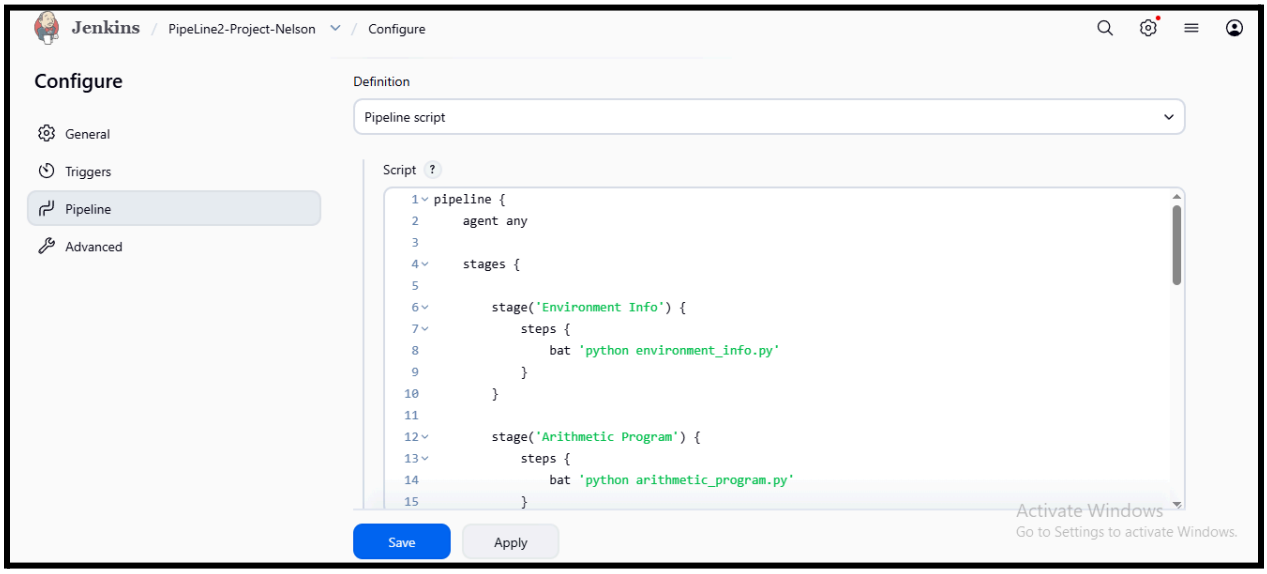
Name: Nelson Puthuraja Nadar **Roll No.:** 21 **Class/Batch:** S.E / INFT-B / B-1

- **Personal Project using Pipeline Script & Running multiple Programs**

Create a Pipeline



Define Pipeline Script -> Pipeline Code




```
pipeline {
   agent any

   stages {

      stage('Environment Info') {
         steps {
            bat 'python environment_info.py'
```

```
        }
    }

    stage('Arithmetic Program') {
        steps {
            bat 'python arithmetic_program.py'
        }
    }

    stage('Factorial Program') {
        steps {
            bat 'python factorial_program.py'
        }
    }

    stage('Prime Number Check') {
        steps {
            bat 'python prime_check.py'
        }
    }

    stage('File Creation Program') {
        steps {
            bat 'python file_creation.py'
        }
    }

    stage('Word Count Program') {
        steps {
            bat 'python word_count.py'
        }
    }

    stage('Loop Demonstration') {
        steps {
            bat 'python loop_demo.py'
        }
    }

    stage('Cleanup') {
        steps {
            bat 'python cleanup.py'
        }
```

```
    }

  }
}
```

What each stage demonstrates:-

1. **Environment Info** – prints system information.
2. **Arithmetic Program** – performs addition and multiplication.
3. **Factorial Program** – calculates factorial using a loop.
4. **Prime Number Check** – checks if a number is prime.
5. **File Creation Program** – creates and reads a file.
6. **Word Count Program** – analyzes file content using wc.
7. **Loop Demonstration** – prints numbers using a loop.
8. **Cleanup** – deletes generated files.

Stage Explanations

• **stage('Environment Info')** → This stage runs a Python program that displays system environment details.

• **steps** → Contains the commands that Jenkins executes during this stage.

• **bat 'python environment_info.py'** → Executes the Python script environment_info.py using the Windows command prompt. The script prints information such as the current user, system directory, or date.

• **stage('Arithmetic Program')** → Executes a Python script that performs basic arithmetic operations.

• **bat 'python arithmetic_program.py'** → Runs the script arithmetic_program.py, which usually calculates and displays values like sum, difference, or product of numbers.

• **stage('Factorial Program')** → Runs a Python program that calculates the factorial of a number.

• **bat 'python factorial_program.py'** → Executes the script factorial_program.py, which uses a loop or function to compute the factorial and prints the result.

• **stage('Prime Number Check')** → Executes a Python program that checks whether a number is prime or not.

• **bat 'python prime_check.py'** → Runs the script prime_check.py, which tests divisibility of the number and prints whether it is prime.

• **stage('File Creation Program')** → Runs a Python script that creates and writes data to a file.

• **bat 'python file_creation.py'** → Executes the script file_creation.py, which generates a file (for example sample.txt) and stores some content inside it.

• **stage('Word Count Program')** → Executes a Python script that counts content inside a file.

• **bat 'python word_count.py'** → Runs the script word_count.py, which reads the file and calculates statistics such as number of lines, words, or characters.

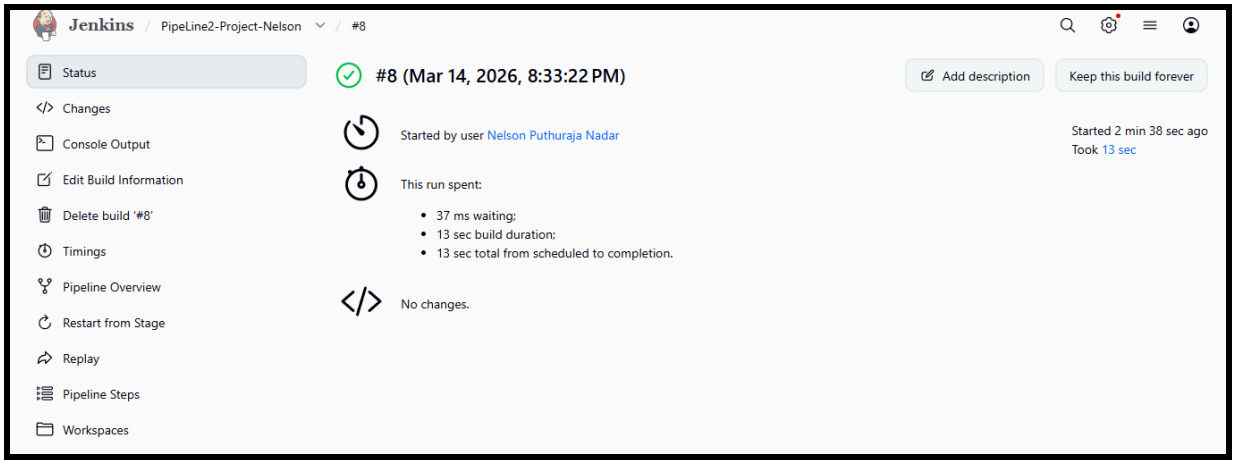
• **stage('Loop Demonstration')** → Runs a Python program that demonstrates loop execution.

• **bat 'python loop_demo.py'** → Executes loop_demo.py, which typically prints numbers or performs repeated actions using a loop.

• **stage('Cleanup')** → Performs workspace cleanup by removing files created during the pipeline.

• **bat 'python cleanup.py'** → Executes cleanup.py, which deletes temporary or generated files to keep the Jenkins workspace clean.

Apply -> Save -> Build Pipeline



Check Console Output

Jenkins / PipeLine2-Project-Nelson / #8

Status
Changes
Console Output
Edit Build Information
Delete build '#8'
Timings
Pipeline Overview
Restart from Stage
Replay
Pipeline Steps
Workspaces

Console

Download Copy View as plain text

```
Started by user Nelson Puthuraja Nadar
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\.jenkins\workspace\PipeLine2-Project-Nelson
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Environment Info)
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\PipeLine2-Project-Nelson>python environment_info.py
=== Environment Information ===
Operating System: Windows
OS Version: 10.0.19045
Machine: AMD64
Current Working Directory: C:\ProgramData\Jenkins\.jenkins\workspace\PipeLine2-Project-Nelson
Current Time: 2026-03-14 20:33:27.469419
Environment check completed.
```

Jenkins / PipeLine2-Project-Nelson / #8

```
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Arithmetic Program)
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\PipeLine2-Project-Nelson>python arithmetic_program.py
=== Arithmetic Program ===
Number A: 15
Number B: 5
Sum: 20
Difference: 10
Product: 75
Division: 3.0
Arithmetic operations completed.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Factorial Program)
[Pipeline] bat
```

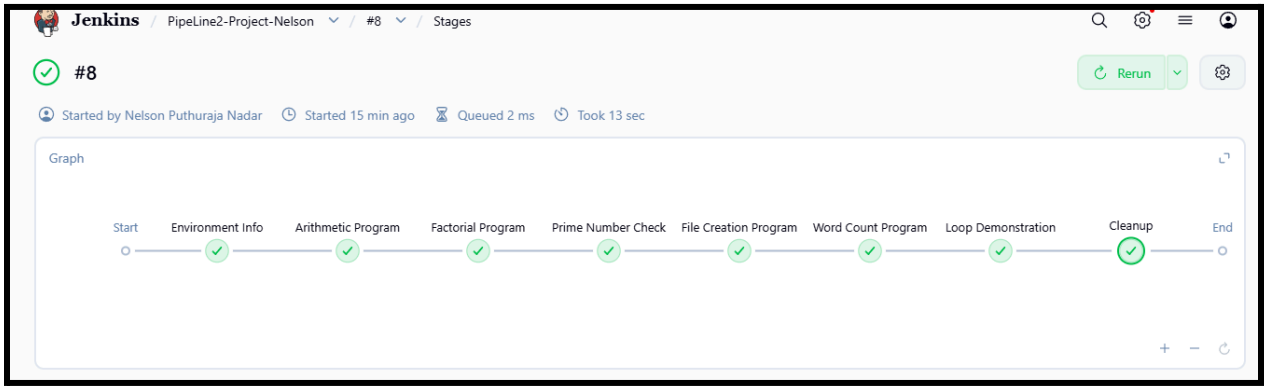
Jenkins / PipeLine2-Project-Nelson / #8

```
C:\ProgramData\Jenkins\.jenkins\workspace\PipeLine2-Project-Nelson>python factorial_program.py
=== Factorial Program ===
Factorial of 5 is 120
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Prime Number Check)
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\PipeLine2-Project-Nelson>python prime_check.py
=== Prime Number Check ===
17 is a Prime Number
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (File Creation Program)
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\PipeLine2-Project-Nelson>python file_creation.py
=== File Creation Program ===
File created successfully.
[Pipeline] }
[Pipeline] // stage
```

Activate Windows
Go to Settings to activate Windows.

localhost:8081

Viewing Pipeline Overview Visualization